# BCA 5th Semester

# JAVA

## Topic : Object-Oriented Programming Concepts

**(PART – 1)**

- By Soudip Sir

| | |
|---|---|
| 3 | **Object-Oriented Programming Concepts:** Access modifiers, encapsulation, and abstraction in Java. Classes, objects, methods, and different types of initialization. Calling a constructor from a constructor, constructor chaining. |
| 4 | **Inheritance and Polymorphism:** Inheritance, method overloading, method overriding, and polymorphism. Restriction on method overriding. |
| 6 | **Interfaces and Abstract Classes:** Understanding interfaces, abstract classes, and multiple inheritance in Java. |

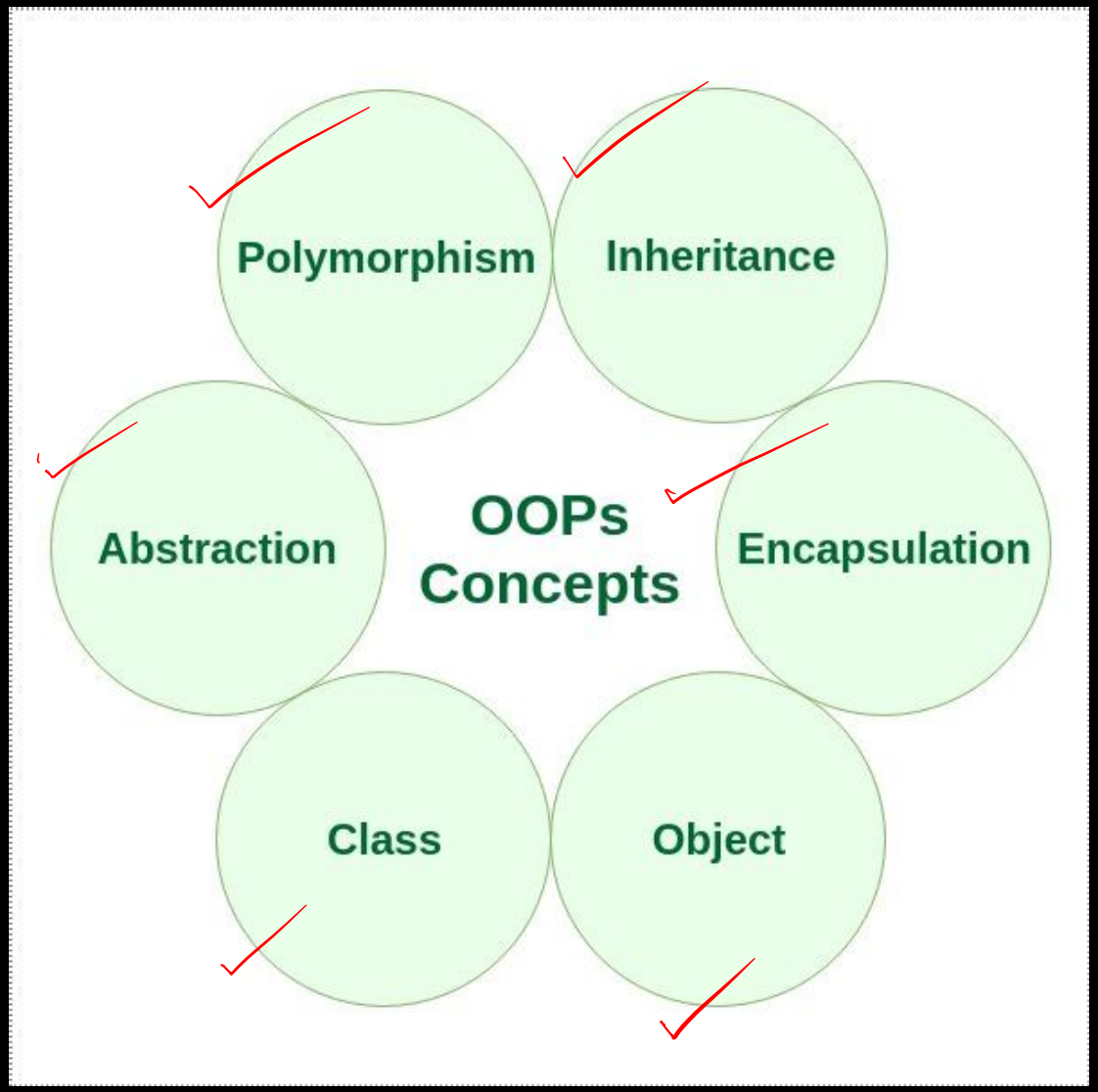
## What is OOPs?

OOPs (Object-Oriented Programming System) is a programming paradigm based on the concept of objects. In OOPs, everything is represented as an object, which contains data (fields/attributes) and methods (functions/behavior).

## Key Features of OOP in Java:

- Structures code into logical units (classes and objects)
- Keeps related data and methods together (encapsulation)
- Makes code modular, reusable and scalable
- Prevents unauthorized access to data
- Follows the DRY (Don’t Repeat Yourself) principle

# OOP Concept in Java:



# 1. Class

A **class** is a blueprint or template used to create objects.

It defines:

- **Data / Attributes** → what an object has
- **Methods / Behaviours** → what an object can do

```
class Student {
    String name;
    int roll;

    void display() {
        System.out.println(name);
        System.out.println(roll);
    }
}
```

Here, Student is a class.

# 2. Object

An **object** is an instance of a class.

Objects are created from classes and contain their own data.

```
Student s1 = new Student();
Student s2 = new Student();
```

Here:

Student → Class

s1 → Object

s2 → Object

new Student() → Creates an object

We can assign different values to each object:

```
s1.name = "Rahul";
s1.roll = 101;

s2.name = "Priya";
s2.roll = 102;
```

## 3. Method

A **method** is a block of code that defines the behaviour or action of an object.

```
class Student {

    String name;

    void study() {
        System.out.println(name + " is studying.");
    }
}
```

void study()

is a **method**.

We can call it using an object:

```
Student s1 = new Student();

s1.name = "Rahul";
s1.study();
```

## 4. Initialization

**Initialization** means giving an initial value to an object's attributes when the object is created.

For example:

```
Student s1 = new Student();

s1.name = "Rahul";

s1.roll = 101;
```

Here, name and roll are being initialized.

In Java, there are several common ways to initialize objects.

# Types of Access Modifiers:

Access specifiers define the scope and visibility of classes, variables, methods, and constructors.

1. public – Accessible from anywhere (inside and outside the package).
2. protected – Accessible within the same package and subclasses (even in different packages).
3. default (no modifier) – Accessible only within the same package.
4. private – Accessible only within the same class.

| Modifier | Class | Package | Subclass | Outside |
|---|---|---|---|---|
| private | ✔ | ✗ | ✗ | ✗ |
| default | ✔ | ✔ | ✗ | ✗ |
| protected | ✔ | ✔ | ✔ | ✗ |
| public | ✔ | ✔ | ✔ | ✔ |

# Encapsulation In Java


Encapsulation is the OOP principle of hiding an object's internal state (its fields) and exposing behavior (methods) to operate on that state. In Java it’s typically done by making fields private and providing public getter/setter methods (or no setter for read-only state).

## Why use encapsulation?

Control: Validate or restrict changes (e.g., prevent negative ages).

Safety: Prevent accidental/unauthorized direct access to internal data.

Flexibility / Maintainability: Implementation can change without affecting callers (API stability).

Encourages invariants: Ensure object remains in a valid state.

## How to implement

- Make fields private.
- Provide public methods (getters/setters) only if needed.
- Validate input inside setters/constructors.
- For mutable internal objects (like lists, Date), use defensive copies or return unmodifiable views.
- Consider immutable objects (no setters, all fields final) where appropriate

Example:

```
class Student {
// private data members (hidden from outside world)
private String name;
private int age;
// public getter method
public String getName() {
return name;
}
// public setter method
public void setName(String name) {
this.name = name;
}
// public getter
public int getAge() {
return age;
}
```

```
// public setter with validation
public void setAge(int age) {
if (age > 0) {
this.age = age;
} else {
System.out.println("Age must be positive!");
}
}
}
```

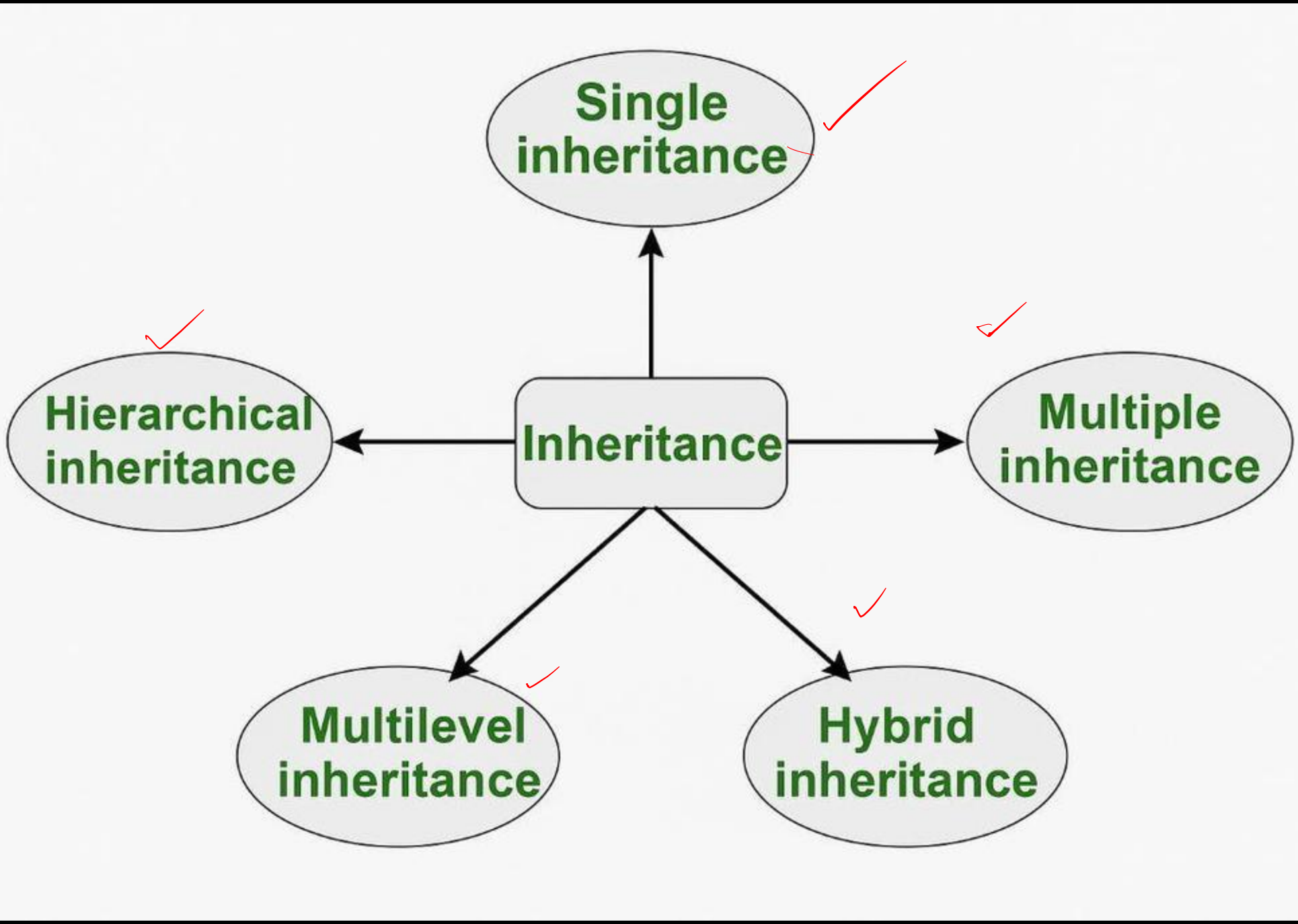
```
public class EncapsulationDemo {
public static void main(String[] args) {
Student s1 = new Student();
s1.setName("Alice");
s1.setAge(20);
System.out.println("Name: " + s1.getName());
System.out.println("Age: " + s1.getAge());
// Trying invalid age
s1.setAge(-5);  // prints warning
}
}
```

# Inheritance In Java

Inheritance is the object-oriented programming (OOP) feature in Java where a class (called child class / subclass) can acquire the properties and behaviours (fields and methods) of another class (called parent class / superclass).

## Key Points

- Achieved using the extends keyword.
- Promotes code reusability.
- Supports method overriding (runtime polymorphism).
- Java does not support multiple inheritance with classes (to avoid ambiguity, "diamond problem"), but it can be achieved with interfaces.


Single inheritance
Hierarchical inheritance
Inheritance
Multiple inheritance
Multilevel inheritance
Hybrid inheritance

# 1. Single Inheritance

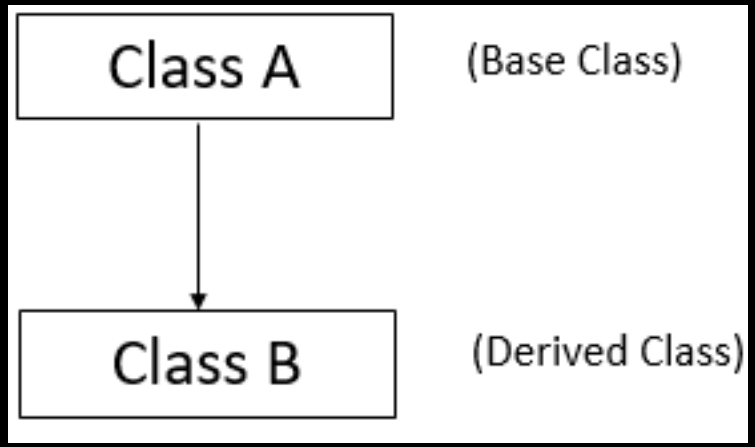
When one child class inherits properties and methods from **one parent class**, it is called single inheritance.

**Key Points:**

•One parent → One child

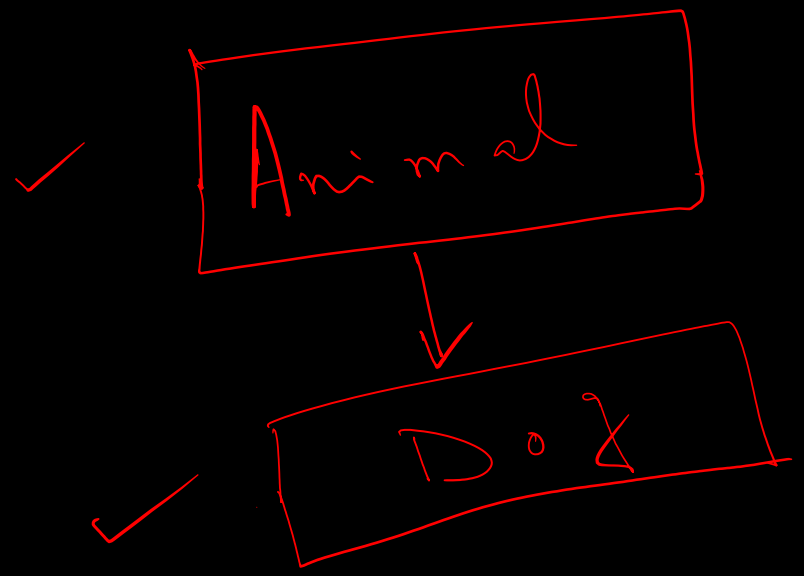
•Simplest form of inheritance

•Promotes code reusability



# Example:

```
class Animal {
void eat() {
System.out.println("This animal eats food");
}
}
class Dog extends Animal {
void bark() {
System.out.println("The dog barks");
}
}
public class SingleInheritanceDemo {
public static void main(String[] args) {
Dog d = new Dog();
d.eat();  // inherited method
d.bark(); // own method
}
}
```



# 2. Multilevel Inheritance

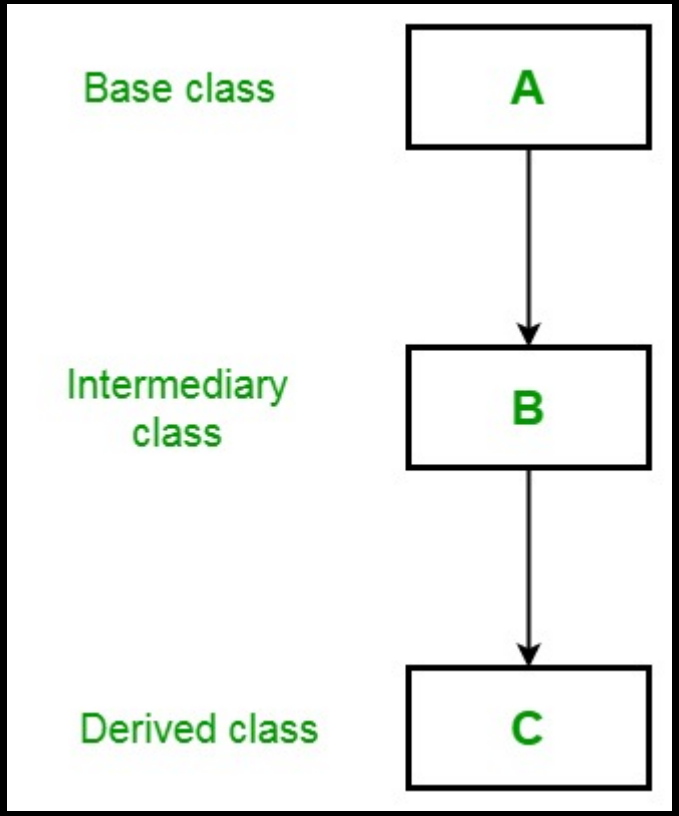
**Definition:** When a class inherits from another derived class, forming a **chain of inheritance**, it is called multilevel inheritance.

**Key Points:**

•Forms a parent → child → grandchild relationship

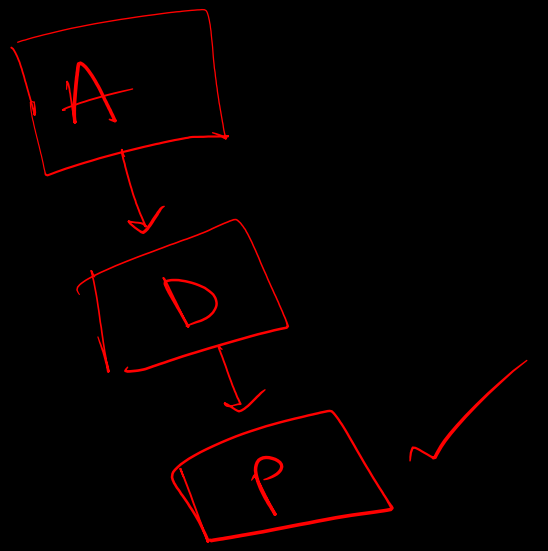
•Properties are passed through multiple levels

•Supports step-by-step specialization



**Example:**

```
class Animal {
void eat() {
System.out.println("This animal eats food");
}
}
class Dog extends Animal {
void bark() {
System.out.println("The dog barks");
}
}
class Puppy extends Dog {
void weep() {
System.out.println("The puppy weeps");
}
}
```




```
public class MultilevelInheritanceDemo {
public static void main(String[] args) {
Puppy p = new Puppy();
p.eat();  // from Animal
p.bark(); // from Dog
p.weep(); // own method
}
}
```

# 3. Hierarchical Inheritance

When **multiple child classes inherit from a single parent class**, it is called hierarchical inheritance.

**Key Points:**

•One parent → Multiple children

•Common properties can be shared among child classes

•Reduces code duplication

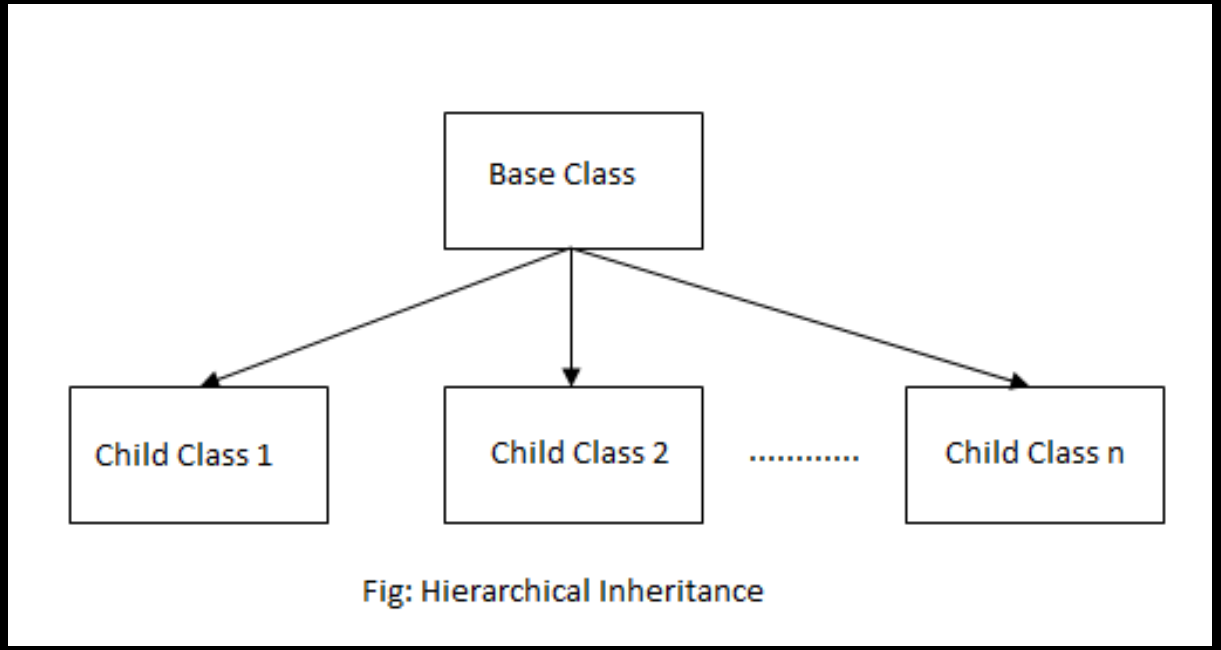
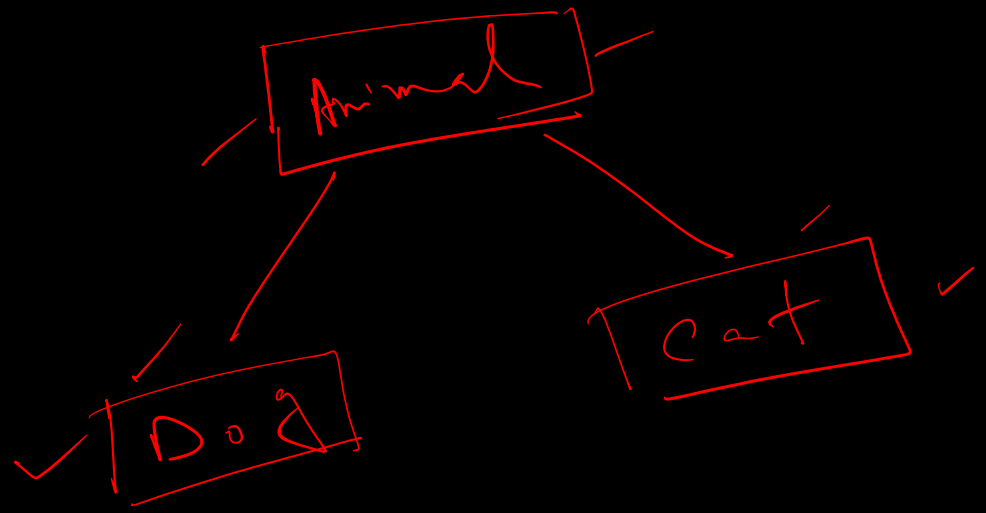



Fig: Hierarchical Inheritance

**Example:**

```
class Animal {
void eat() {
System.out.println("This animal eats food");
}
}
class Dog extends Animal {
void bark() {
System.out.println("The dog barks");
}
}
class Cat extends Animal {
void meow() {
System.out.println("The cat meows");
}
}
```




```
public class HierarchicalInheritanceDemo {
public static void main(String[] args) {
Dog d = new Dog();
d.eat();
d.bark();
Cat c = new Cat();
c.eat();
c.meow();
}
}
```

# 4. Multiple Inheritance

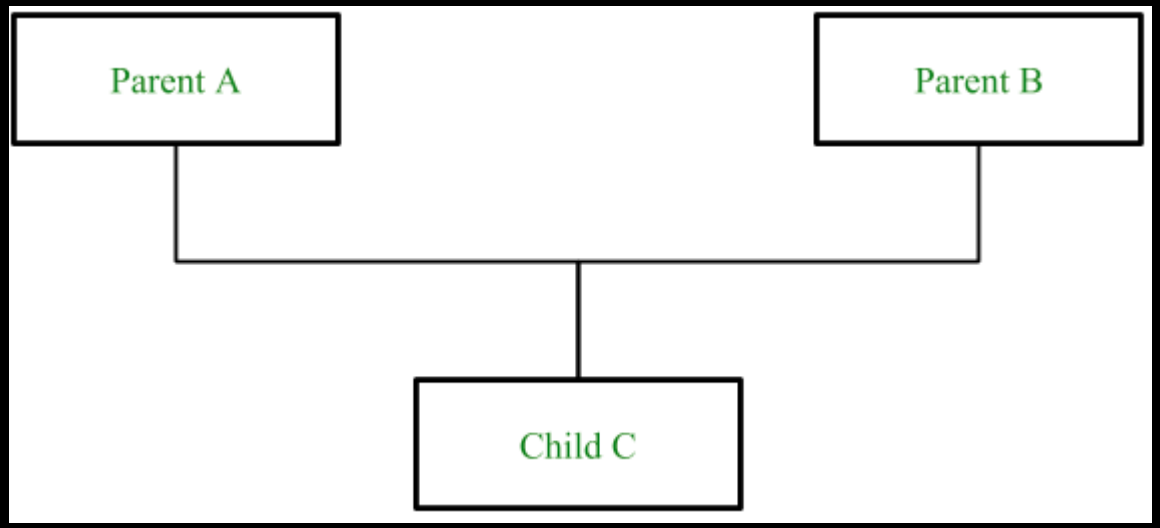
When one child class inherits properties and methods from **more than one parent class**, it is called multiple inheritance.

**Key Points:**

•Multiple parents → One child

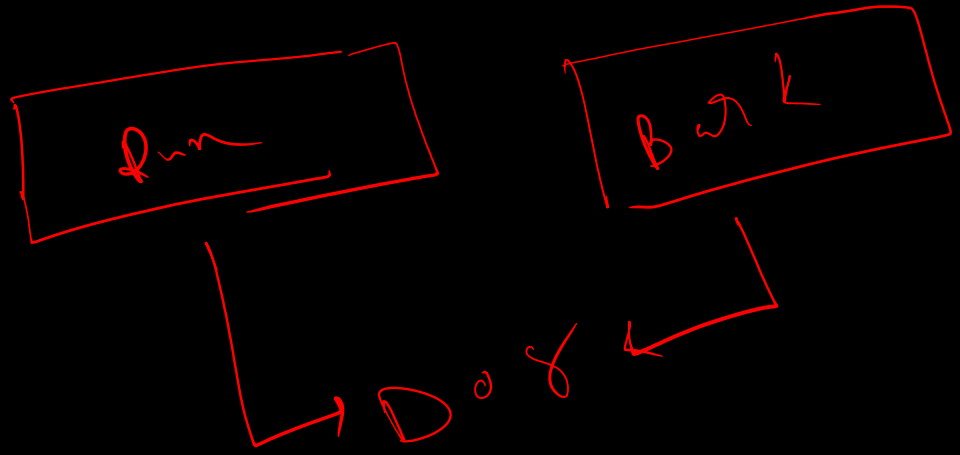
•Allows combining features of different classes

•**Java does not support multiple inheritance through classes** to avoid ambiguity



**Example:**

```
interface CanRun {
void run();
}
interface CanBark {
void bark();
}
class Dog implements CanRun, CanBark {
public void run() {
System.out.println("The dog runs fast");
}
public void bark() {
System.out.println("The dog barks loudly");
}
}
```




```
public class MultipleInheritanceDemo {
public static void main(String[] args) {
Dog d = new Dog();
d.run();
d.bark();
}
}
```

# 5. Hybrid Inheritance

When **two or more types of inheritance are combined** together, it is called hybrid inheritance.

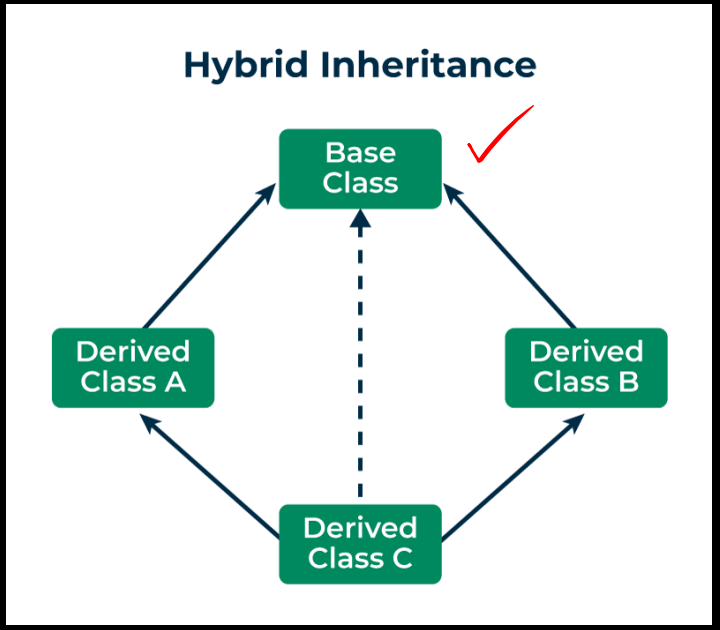
**Key Points:**

•Combination of different inheritance types

•Can create complex class relationships

•In Java, it can be achieved using **interfaces** rather than multiple inheritance of classes

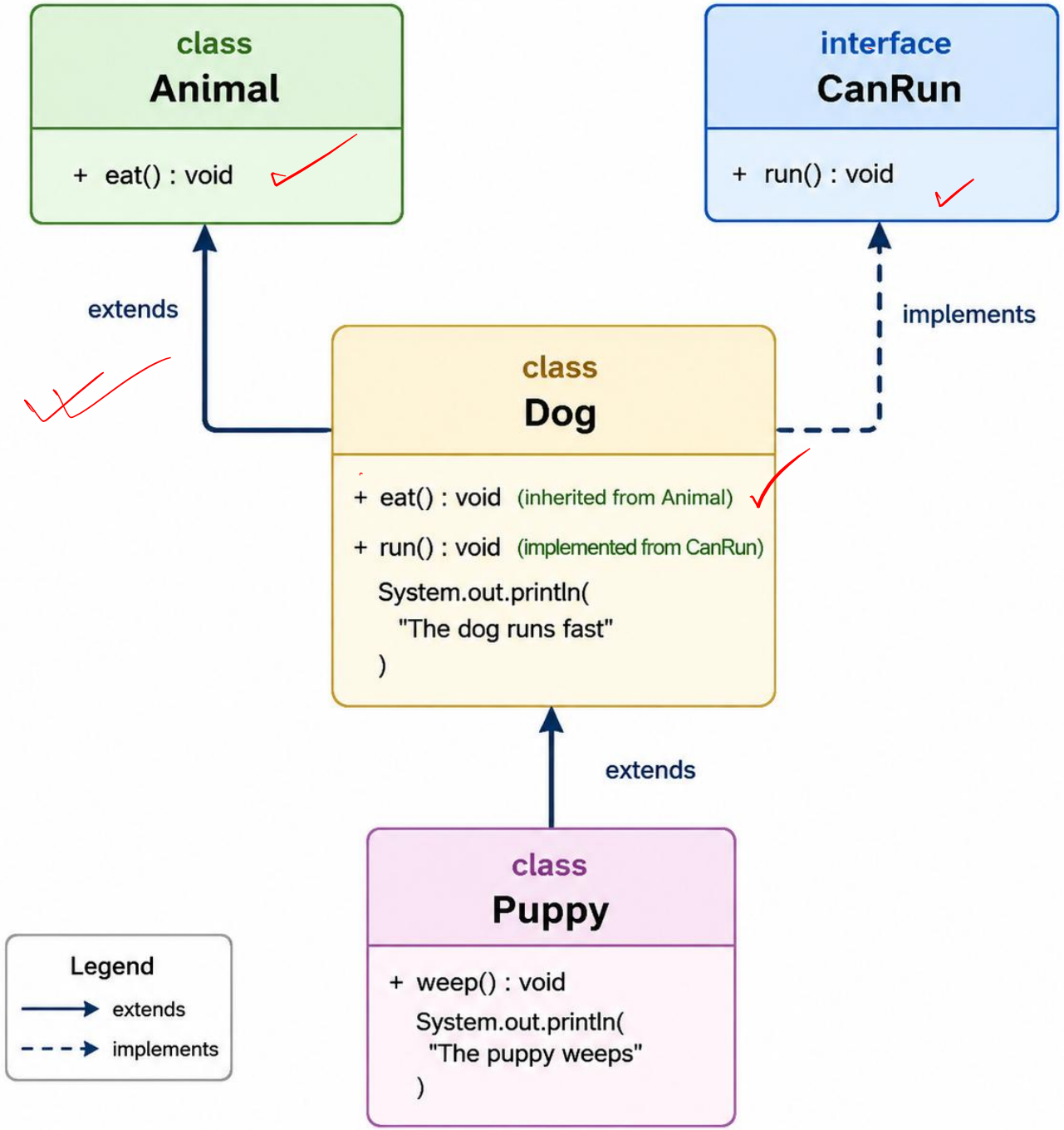
**Example:** Combination of **multilevel + hierarchical inheritance**.



**Example:**

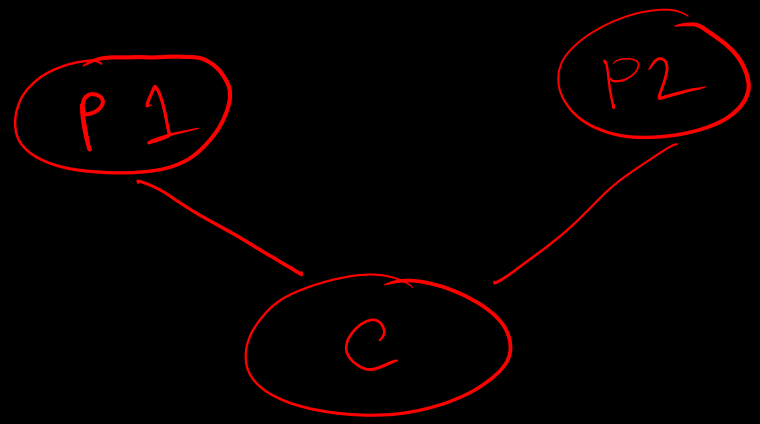
```
interface CanRun {
void run();
}
class Animal {
void eat() {
System.out.println("This animal eats food");
}
}
class Dog extends Animal implements CanRun {
public void run() {
}
System.out.println("The dog runs fast");
}
class Puppy extends Dog {
void weep() {
System.out.println("The puppy weeps");
}
}
```

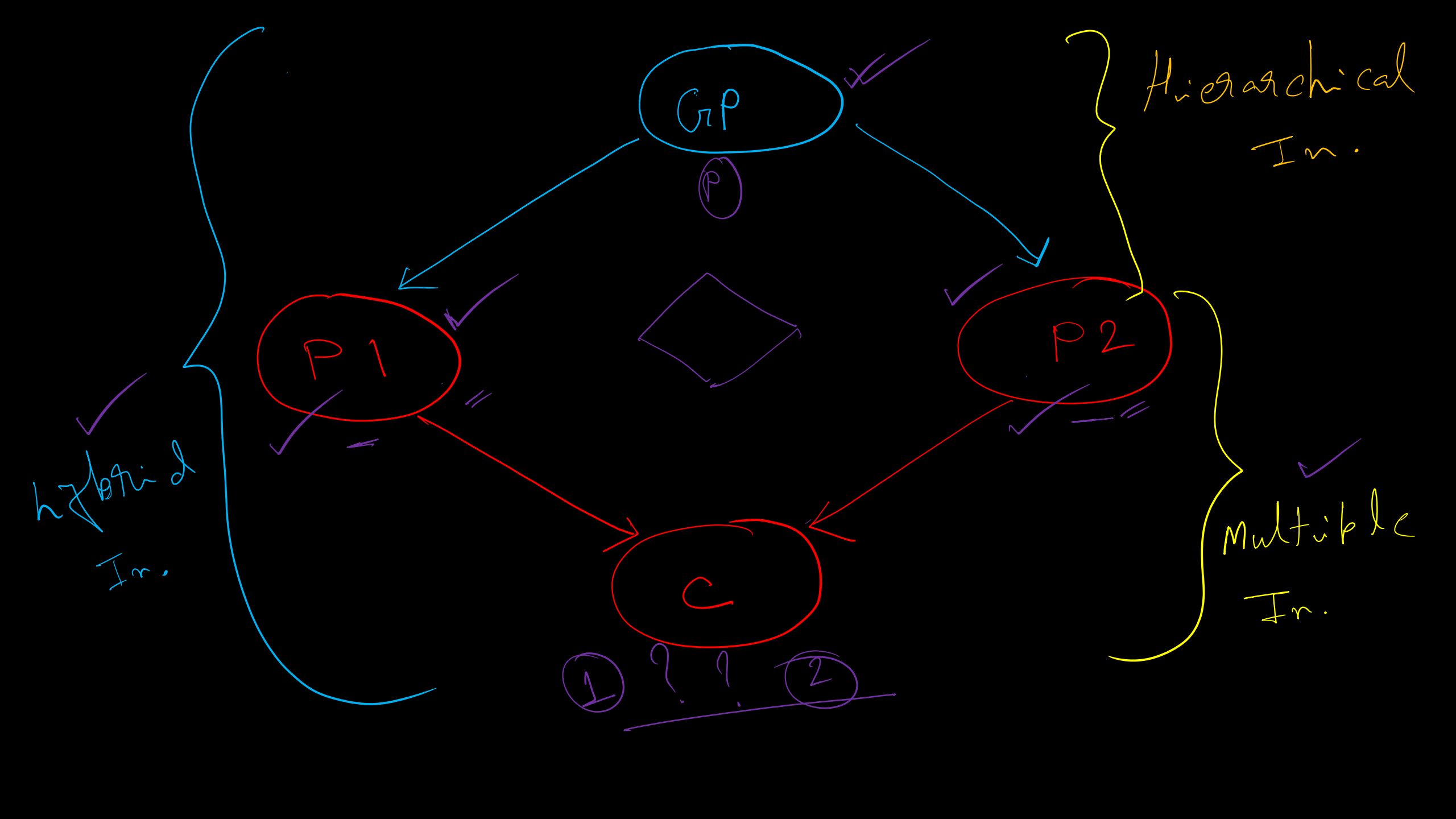
```
public class HybridInheritanceDemo {
public static void main(String[] args) {
Puppy p = new Puppy();
p.eat();   // from Animal
p.run();   // from CanRun (interface)
p.weep();  // own method
}
}
```


class
Animal
+ eat() : void
interface
CanRun
+ run() : void
extends
implements
class
Dog
+ eat() : void (inherited from Animal)
+ run() : void (implemented from CanRun)
System.out.println(
"The dog runs fast"
)
extends
class
Puppy
+ weep() : void
System.out.println(
"The puppy weeps"
)
Legend
extends
implements

## Diamond Problem in Java

- The Diamond Problem occurs in multiple inheritance when a class inherits from two classes that have a common parent class.
- This creates ambiguity about which parent's method or property should be inherited by the child.
- The inheritance diagram looks like a diamond shape, hence the name.




GP
P1
P2
C
Hierarchical In.
Hybrid In.
Multiple In.

A.W

1) Encapsulation → Menu + Input

2) Inheritence → 5

{ Intaractive → Input
We Scanner class }